

Exercise 3 (Summer 2026)

6.0 VU Advanced Database Systems

Information

General

Work through the exercises below and write a report on your solutions. Submit the report and the other required files as a **single ZIP (.zip)** file on TUWEL. The ZIP (.zip) file must contain:

- one PDF (.pdf) file,
- the JSON files for Exercise 2.

The ZIP (.zip) file must be at most 20 MB in size. After submitting the files, you need to register for an exercise interview. **You can only receive points for this exercise if you attend the interview.** You are expected to discuss your solutions and answer questions regarding them in your report. **We will not accept any handwritten reports!**

This exercise sheet contains 4 exercises, with a total of 10 points achievable.

Deadlines

at latest June 16th 9:00 Upload your submission on TUWEL
Do not forget! ⇒ Register for an exercise interview in TUWEL

Exercise Interviews

During the exercise interview, the correctness of your solution as well as your understanding of the underlying concepts will be assessed. The scoring of your submission is primarily based on your performance at the exercise interview. Therefore it is possible (in extreme cases) to get 0 points even though the submitted solution was technically correct.

Please be punctual for your exercise interview. Otherwise we cannot guarantee that your full solution can be graded in your assigned time slot. **Remember to have your student id with you for the exercise interview.** It is not possible to receive a score for your solution without showing your id, as we cannot confirm your identity.

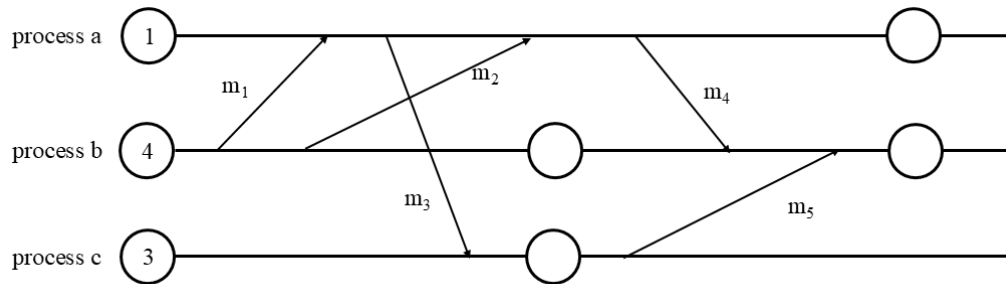
TUWEL Forum

You can use the course forum on TUWEL for clarifying questions regarding the exercise sheets. Please do not post your solutions (even partial) on the forum.

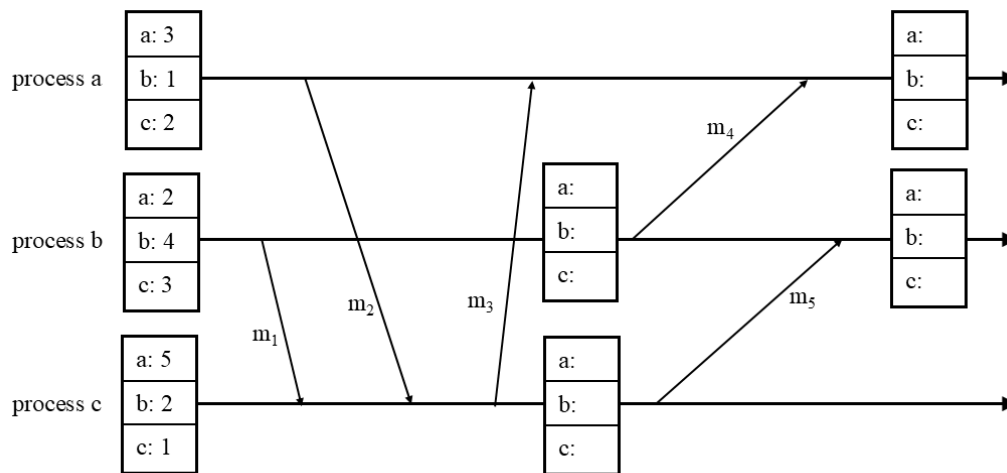
Exercises

Exercise 1 (Scalar and Vector Clocks) [2 points] Below, you are given 3 processes exchanging messages between each other and maintaining version stamps. The system may comprise further processes, but we are not interested in them here. The version stamps are in the form of Scalar clocks (part a) and Vector clocks (part b), respectively. In both cases, you are given the initial values of the version stamps for each of the 3 processes. Your task will be to fill in the missing values of the version stamps at some specified “points in time”.

- (a) Please note that we observe the 3 processes only for a small time window. The processes have already started before. Therefore, at the beginning of the time window, the clock counts are not 0. Instead, they are 1, 4 and 3, respectively. Within this time window, 5 messages $m_1, \dots, m_5$ are sent from one of these processes to another one. It is your task to fill in the correct clock values into the 4 empty circles, indicating the current value of the clock at this particular point in time. Please also fill in all clock count values before/after sending/receiving messages.



- (b) Please note that we observe the 3 processes only for a small time window. The processes have already started before. Therefore, at the beginning of the time window, the vector clocks have values different from (0, 0, 0). Within this time window, 5 messages $m_1, \dots, m_5$ are sent from one of these processes to another one. It is your task to fill in the correct clock values into the 4 empty rectangles (each with 3 cells for the 3 processes that we want to observe here), indicating the current value of the clock at this particular point in time. Please also fill in all vector clock values before/after sending/receiving messages.



Exercise 2 (Denormalization) [2 points] Consider a hotel room booking system implemented in a traditional RDBMS and assume that you want to migrate this system to use a document store using JSON representation instead. The relational model has 4 tables **room**, **booking**, **occupancy**, **guest**; an example instance (the key is underlined in each relation) is given below. Conceive a data model for your document store system and translate the exam-

ple instance to JSON files fitting your data model. You are allowed to add new attributes, denormalize relations, and make other similar changes for your new data model.

As discussed in the lecture, data models are designed with specific workloads in mind. Trade-offs may be necessary; if so, describe them in your report. For this exercise consider the following parameters:

- The following query should be fast:
 - Listing the names of guests who stayed at the hotel in a given year.
- The **booking** relation is updated frequently (and needs to be efficient w.r.t. updates). For all other relations, updates are rare and can be more expensive.

room		
<u>id</u>	type	price
1	Single	89
2	Double	149
3	Suite	299

booking			
<u>guest</u>	<u>room</u>	nights	<u>check_in</u>
3	1	3	05-10-2025
12	2	7	12-10-2024
15	3	2	20-03-2025
15	3	5	30-09-2024
18	1	10	01-03-2025

occupancy		
<u>room</u>	<u>year</u>	available_nights
1	2024	280
1	2025	310
2	2025	240
3	2024	180
3	2025	195

guest	
<u>id</u>	name
3	Bob
12	Alice
15	Tom
18	Ben

Your submission for this exercise should have two parts: a **description of your data model** in your report and the **resulting JSON files**.

Exercise 3 (Graph Databases) [4 points] For this exercise you will familiarize yourself with Neo4j and the Cypher query language to experience the basics of graph databases. The lecture on graph databases contained an introduction to Cypher. Beyond that, we recommend the official tutorial¹ as a supplement, should you need one.

The exercise uses a subset of the Wikidata knowledge graph (<https://www.wikidata.org>). Wikidata is a large open (public domain) collection of linked data which can be collaboratively edited, similar to (and connected to) Wikipedia, but in a more structured format.

The Dataset

The database contains many computer scientists, as well as persons in general for whom an entry in Wikidata exists.

¹<https://neo4j.com/docs/cypher-manual/current/introduction/>

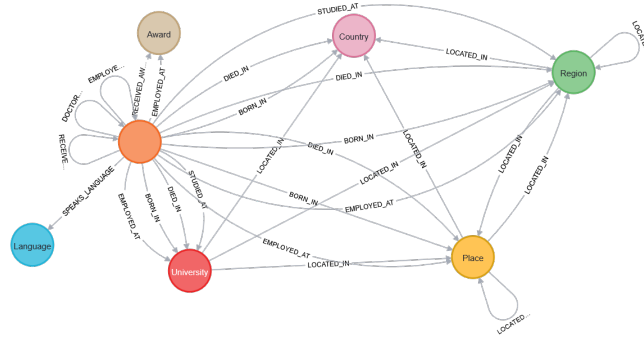


Figure 1: The data model (call `db.schema.visualization()`)

Persons can work (**EMPLOYED_AT**) or study at (**STUDIED_AT**) zero or more *Universities*. Universities are located in (**LOCATED_IN**) a *Country*. Persons are also born in (**BORN_IN**) a *Place*. A Place is located in (**LOCATED_IN**) a *Region*, which might again be located in a region. Persons can also receive (**RECEIVED_AWARD**) zero or more *Awards*. Persons can speak (**SPEAKS_LANGUAGE**) (as far as known) languages (*Language*).

Note that the relationships between node types are not entirely consistent. For example, in two cases the *reward* relation occurs between two Persons. This is a limitation of Wikidata. If anything is unclear, make plausible assumptions.

Running Neo4j

1. Make sure docker and docker-compose are installed.
2. Download the zip archive linked in TUWEL and unpack it.
3. In the unpacked directory, run `sudo docker-compose up`.
4. Open `http://localhost:7474/browser/` and log in with the username `neo4j` and the password `neo4jneo4j`.
5. To completely remove the containers, use `sudo docker-compose down`. Note that this removes the docker volumes as well, where the data of the DB is stored (save your queries!).

Your Tasks



Figure 2: Export as Image from Neo4j Browser

Provide the requested queries in your solution and where sensible also provide pictures of the output graph. The Neo4j browser has built-in support for exporting the graph as an image (cf. Figure 2).

- i. List all the *distinct* languages that are spoken by at least one person who has also received at least one award.

- ii. List all the *distinct* countries where a university is located in which any person has studied. **Note:** you can use the `LOCATED_IN` relation between *University* and *Country*.
 - iii. Extend the list from (ii) by how many persons have studied at the universities in this country. Your output should thus consist of pairs of countries and the number of persons that have studied at a university in this country.
 - iv. Extend the output from (iii) by the following information: for each country, show the count of persons who studied at universities in this country *and* who speak at least two different languages. Order the output by this count.
2.
 - i. Find the 10 **top universities** by the number of persons employed there, i.e. those 10 universities that have the most incoming `EMPLOYED_AT` edges. Output the name of the university as well as the number of relevant edges. Order the output in *descending* order by the number of incoming `EMPLOYED_AT` edges.
 - ii. Places are contained in (`LOCATED_IN`) *Regions*, which represent administrative areas (e.g., “Greater London”). Regions may again be contained in Regions, and a Place might also be a Region.

Find the top 10 regions by the number of persons. **Note:** it is not required to consider the case that regions and places correspond — you can just choose the region one level above the place of birth.
 - iii. Find the top 3 countries by the number of persons who were employed at one of the universities there (`EMPLOYED_AT`) at one point, who were awarded both “Turing Award” and “ACM Fellow” award.
3. Find one subgraph of the form specified in Figure 3 in the database.

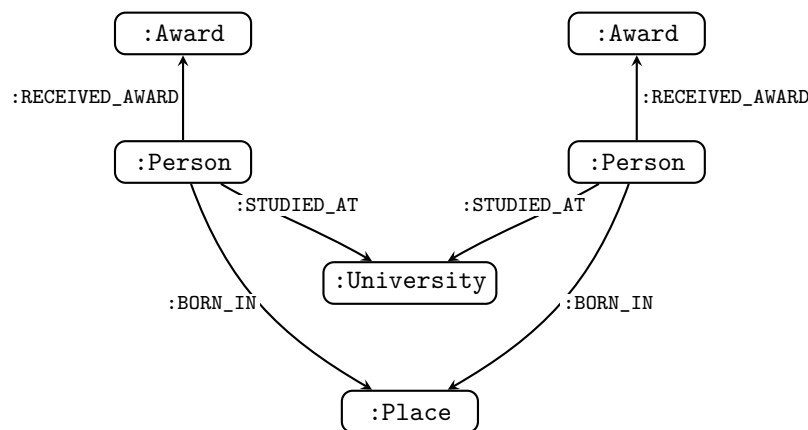


Figure 3: Subgraph for Query

4. We want to make the `last_name` and `date_of_birth` attributes a real part of the graph. The best way to do this in Neo4j is using the `MERGE` keyword inside of a `MATCH`.
 - First, recall what you learned about the `MERGE` keyword in the lecture. As a supplementary source of information about `MERGE`, we recommend the documentation: <https://neo4j.com/docs/cypher-manual/current/clauses/merge/>.
 - Write a query that for each node `p` with label `Person`, of a person who has **received at least one award**, performs the following actions:

- Add an edge with type `HAS_NAME` going to a node with label `Name` that has a `name` attribute matching the contents of `p.name`.
- If the node `p` has no `name` attribute, do nothing for this node.
- Add an edge with type `HAS_DOB` going to a node with label `DateOfBirth` that has a `date_of_birth` attribute matching the contents of `p.date_of_birth`.
- If the node `p` has no `date_of_birth` attribute, do nothing for this node.
- Make sure that you **don't create duplicate nodes or duplicate edges**.
- When done, write a query to show all the `Person` nodes connected to the 5 most common last names among the persons processed above.

Hint: If you have created wrong entries while experimenting, you can delete them using:

```
MATCH (n:Name) DETACH DELETE n;
```

5. Consider the graph extended with the following additional relationship: for any two universities u and v , for which two *different* persons p_1 and p_2 exist, where p_1 studied at both u and v and p_2 was employed at both u and v , u and v are connected. On the extended graph, find *shortest paths* between the university “TU Wien” and *all* universities located in the country “Germany”. Neo4j provides the `shortestPath` function for these types of problems. Be aware that this will not return the singular shortest path between any nodes. Rather, `shortestPath` returns a shortest path for each match.

Note that it might be convenient as an intermediate step to introduce new edges between the `University` nodes.

Exercise 4 (Column Stores: Late Materialization Query Plan) [2 points]

Consider the following query executed on a **column store** that uses **late materialization**:

```
SELECT sum(R.a) FROM R, S WHERE R.b = S.a AND 5 <= R.a AND R.a <= 20
```

There are two relations R and S . Each column is represented as a list of (*position*, *value*) pairs:

Relation R

Ra		Rb	
<i>pos</i>	<i>val</i>	<i>pos</i>	<i>val</i>
1	3	1	12
2	16	2	34
3	25	3	17
4	9	4	56
5	5	5	22
6	31	6	34

Relation S

Sa		Sb	
<i>pos</i>	<i>val</i>	<i>pos</i>	<i>val</i>
1	34	1	7
2	22	2	4
3	17	3	9

Query Plan

The query is evaluated using the following query plan. All operations work on single columns.

```

1. inter1 = select(Ra, 5, 20)
2. inter2 = reconstruct(Rb, inter1)
3. join_input_S = reverse(Sa)
4. join_res    = join(inter2, join_input_S)
5. inter3 = voidTail(join_res)
6. inter4 = reconstruct(Ra, inter3)
7. result    = sum(inter4)

```

Operation reference:

- `select(col, low, high)` – returns a position list (list of IDs) of all entries in `col` where $low \leq val \leq high$.
- `reconstruct(col, poslist)` – fetches values from `col` at each position in `poslist`; returns (pos, val) pairs.
- `reverse(col)` – swaps each (pos, val) to (val, pos) ; used to build a lookup structure where value is key.
- `join(left, right)` – joins `left` values against `right` keys; returns (pos_L, pos_R) pairs.
- `voidTail(pairs)` – keeps only the left element of each pair.

Task: Fill in the Intermediate Results

Please fill in the empty cells after each step. The first row indicates the names of the returned columns, as shown in Step 1 and Step 2. The number of blank light-grey rows indicates the expected number of result tuples.

Step 1: `inter1 = select(Ra, 5, 20)`

<i>pos</i>

Step 2: `inter2 = reconstruct(Rb, inter1)`

<i>pos</i>	<i>Rb</i>

Step 3: `join_input_S = reverse(Sa)`

Step 4: `join_res = join(inter2,`
`join_input_S)`

Step 5: `inter3 =`
`voidTail(join_res)`

Step 6: `inter4 = reconstruct(Ra,`
`inter3)`

Step 7: `result = sum(inter4)`

result